Contents

1 fouine : généralités, en facteur de ce qui suit **1**

2 Les étapes de la croissance de fouine **2**
2.1 Cœur du langage fouine . . . 2
2.1.1 Fonction `prInt` . . . 3
2.2 Déclarations et variables . . . 4
2.2.1 La construction `let.. in` : formes dérivées. . . . 4
2.3 Fonctions, fonctions récursives . . . 4
2.3.1 Fonctions récursives . . . 5
2.4 Références et aspects impératifs . . . 5
2.5 Exceptions . . . 6
2.5.1 Comment ça marche en Caml . . . 6
2.5.2 Comment traiter les exceptions en fouine . . . 8
2.6 Couples, motifs, tuples . . . 8
2.7 Listes . . . 9
2.8 Types algébriques . . . 9

3 Indications pour les analyses lexicale et syntaxique **9**
3.1 Corriger des bugs dans le parser . . . 10
3.2 Analyse des applications . . . 10
3.3 Séquence et listes. . . . 11

4 Ajouter du typage — inférence de types **13**
4.1 Inférence de types monomorphe . . . 13
4.2 Le polymorphisme . . . 14

5 Continuations **17**
5.1 Programmation par continuations, principes . . . 17
5.2 Implémenter les exceptions à l'aide de continuations . . . 17
5.3 Traduire pour éliminer les exceptions . . . 17
5.4 Programmer la traduction . . . 19

6 pieuvre, un petit assistant de preuve **20**
6.1 Mise en place : λ-calcul, réduction, typage . . . 20
6.1.1 λ-calcul, β-réduction. . . . 20
6.1.2 Types. . . . 21
6.2 Manipulation d'une preuve . . . 21
6.3 Tactiques . . . 23
6.4 Exécutable, session . . . 23
6.4.1 Exemples d'entrées à soumettre à pieuvre . . . 24
6.5 Enrichir le langage : `/\` `\/` `True` . . . 25
6.6 Encore plus loin : types dépendants . . . 26

1 fouine : généralités, en facteur de ce qui suit

fouine comme sous-ensemble de Caml. On décrit dans ces notes le langage fouine. Dans ce qui suit, on utilise "fouine" pour désigner à la fois le sous-langage de Caml sur lequel vous travaillez et l'interprète que vous devez implémenter — on fait d'ailleurs de même avec "Caml" (et pour nous, Caml=OCaml).

Chaque fois que vous vous demandez si fouine doit faire comme ci ou comme ça, écrivez des tests que vous ferez tourner sur Caml. Cela vous donnera la réponse. Il faut bien sûr garder le ou les tests que vous

avez écrits suivant cette approche, vous pourrez les ajouter aux autres tests que vous ferez subir à votre fouine.

Différences fouine/Caml :

- fouine connaît une fonction `prInt`, qui n'est pas définie en Caml (cf. ci-dessous). Pour tester votre fouine, on ajoutera la définition de `prInt` afin de pouvoir comparer les exécutions en Caml et en fouine.
- fouine n'est pas typée : le programme `if true then` 5 `else` (`fun` $x \to x+1$) est accepté en fouine, et renvoie `5` sans aucun froncement de sourcil. L'ajout de l'inférence de types est traitée plus loin.
- fouine n'accepte que des fichiers contenant *une seule* expression (qui peut être un `let .. in ..`, bien sûr). Pas de `3;; 5;; 7` en entrée de fouine.

Saisie et affichage : fouine accepte et affiche des programmes Caml. Le principe est que tout programme utilisant le sous-ensemble de Caml que traite votre fouine doit être accepté, *tel quel*, par fouine. Vous ne devez pas adopter une syntaxe différente, imposer l'usage de parenthèses supplémentaires, ou autre.

Inversement, une option de votre fouine affichera le code qui a été saisi. L'affichage devra produire du code accepté par Caml. À noter que le code affiché ne sera pas nécessairement identique au code saisi.

On vous encourage à vous appuyer sur les fonctions de la librairie `Format` de Caml pour programmer cette fonctionnalité (vous pouvez aussi consulter des tutoriels en ligne sur cette librairie).

Tests, messages d'erreur. Nous fournissons une batterie de tests. Votre fouine est sensée se comporter comme Caml sur ces tests.

Vous devez également inventer des tests vous-même, en particulier pour fouiller "dans les coins" pour mettre en évidence des comportements particuliers des divers éléments de Caml que vous traitez, et les interactions que ces éléments peuvent avoir entre eux.

Lorsque fouine échoue, parce que le programme en entrée est incorrect, ou engendre un comportement incorrect, engendrez, autant que faire se peut, un message d'erreur pertinent. Une bonne approche pour ce faire est de déclarer diverses exceptions, correspondant aux divers types d'erreurs que l'on peut rencontrer (mécanisme `exception` / `raise` de Caml).

2 Les étapes de la croissance de fouine

2.1 Cœur du langage fouine

Dans le cœur de fouine, il y a :

- les constantes entières (y compris négatives) et les booléens
- les opérateurs `+ - * / >= = > < <> <= || && not`
- les parenthèses, ainsi que la construction `begin end`, qui sont synonymes—mais ne peuvent pas dialoguer : on ne ferme pas un `(` avec `end`.
- les sauts à la ligne dans le code fouine sont possibles, et n'affectent pas la compréhension des programmes (mais on ne saute pas à la ligne au milieu d'un mot-clef : pas de `th` suivi de `en` à la ligne suivante si on veut écrire `then`).
- la construction `if then else`. Les booléens existent en fouine, et fouine plante si on tente d'exécuter `if 3 then 5 else 7`. "Plante" au sens où fouine interrompt son exécution et affiche un message d'erreur. Suivant la manière dont vous implémentez fouine, le plantage peut advenir au moment où le programme est évalué, ou alors avant. Mais en première approche, fouine ne vérifie pas le typage dans les programmes que l'on saisit. En particulier, `if true then` 5 `else` (`fun` $x \to x+1$) renvoie `5` sans problème.

  On a `if then else`, mais pas de `if then` tout court.

Opérateurs binaires. Plutôt que d'avoir un constructeur fouine par opérateur binaire (addition, multiplication, supérieur ou égal, conjonction, etc.), pensez à factoriser[1].

Factoriser permet d'éviter la duplication du code, et de limiter le nombre de constructeurs pour le type `expr`.

À noter qu'en fouine, comme en Fun, tout est `expr` : la grammaire n'est pas stratifiée en expressions booléennes et arithmétiques et commandes, mais "à plat".

Les erreurs que ne connaît pas Caml. En Caml, un programme est typé, ce qui permet d'éviter d'exécuter des programmes absurdes comme `3 + (fun x -> x+1)`. Pas de typage en fouine, par conséquent certains programmes exploseront lors de l'exécution.

Réfléchissez à gestion de ces erreurs (appelées *dynamiques*), et proposez une solution satisfaisante. Cela peut aller de messages d'erreur un peu explicatifs dans certains cas (quand une addition fait intervenir une fonction, par exemple) à des choses plus sophistiquées, où l'on indique d'où vient l'erreur dans le code source.

Pour la gestion des erreurs, vous pouvez définir dans un endroit judicieux de votre code la définition des erreurs, de la forme

```
exception DivisionParZero

exception IlEstTropTot of string

...
```

ce qui vous permettra de faire `raise DivisionParZero` ou `raise (IlEstTropTot "il est midi")` aux endroits où l'on veut signaler une erreur et s'arrêter.

Adressez-vous aux encadrants, ou cherchez en ligne, si vous avez besoin d'explications s'agissant de la manipulation de `exception`, `raise` et `try.. with`.

2.1.1 Fonction `prInt`

On ajoute une construction spéciale au langage, pour pouvoir faire de l'affichage et ainsi voir ce que font les programmes. Cette construction s'appelle `prInt`, c'est une fonction qui affiche son argument entier, va à la ligne, et renvoie pour valeur ce même argument. En Caml (mais pas en fouine), on peut définir `prInt` comme suit :

```
# let prInt x = print_int x;print_newline(); x;;
val prInt : int -> int = <fun>
```

Et voici un petit exemple :

```
# 2 + prInt (1+2);;
3
- : int = 5
```

Votre fouine devra avoir le même comportement sur un fichier d'entrée `toto.ml` (y compris en termes de nombres de retours à la ligne) qu'aurait le compilateur Caml sur le fichier `toto.ml` auquel on aurait ajouté au début la définition de `prInt` donnée ci-dessus. Par conséquent, au moment d'ajouter `prInt` à votre fouine, modifiez le fichier `bin/fouine.ml`, de manière à ce que fouine n'affiche *que ce qui est affiché par* ***`prInt`*** : finis les affichages superflus, on colle à Caml (ce qui facilite le test de vos fouines).

À noter qu'en présence d'affichage, *l'ordre d'évaluation compte* : à vous de concevoir des tests permettant de vérifier que votre interprète se comporte comme Caml.

[1] Si votre binôme est avancé, vous pouvez regarder la partie "Generalized algebraic datatypes" du manuel de Caml — mais cela est facultatif, et ne perdez pas de temps là-dessus si cela est obscur pour vous.

2.2 Déclarations et variables

Il s'agit d'ajouter la construction `let .. in`, et la notion d'environnement (cf. les transparents du premier cours).

Les *identificateurs de variables* commencent par une lettre minuscule ; après, n'importe quoi en termes de chiffres et de lettres (majuscules ou minuscules) ; `let oOOOo = 0` marche, par exemple.

Les déclarations sont un des cas de la grammaire de fouine, à part entière ; il ne faut pas traduire une déclaration en une application (notamment pour des raisons de typage, lorsque vous ajouterez l'inférence de types).

On vous conseille d'avoir d'abord une version qui marche avec les `let.. in` simples, avant de vous attaquer aux formes dérivées ci-dessous.

2.2.1 La construction `let.. in` : formes dérivées.

On vous demande d'autoriser deux écritures particulières pour les déclarations.

N.B. : cela se traite au niveau de l'analyse lexicale et syntaxique. Dans le type des expressions de votre interprète, il faut avoir une seule construction pour `let.. in`.

ne pas nommer Vous devez autoriser l'utilisation du caractère "souligné" tout seul, dans des expressions de la forme `let _ = ... in ...`

Cette utilisation sert à ne pas nommer une expression que l'on évalue (on l'utilise généralement lorsqu'on fait des effets de bord – affichage à l'écran, écriture dans un fichier, mise à jour de référence ..).

utilisation de ;; On peut remplacer un `in` par `;;` uniquement dans un `let.. in` "en surface". Dans ce cas, tous les `in` dans les déclarations qui précèdent doivent être remplacés par des `;;`.

Un programme est de la forme `let x1 = e1 in let x2 = e2 in... let xn = en in e`. Les déclarations des `xi` sont dites "en surface", mais un `let.. in` qui se trouve dans l'un des `ei` n'est pas en surface. Si on écrit pour un tel programme `let x4 = e4;; let x5 = ..`, alors il faut nécessairement écrire avant `let x1 = e1;; let x2 = e2;; let x3 = e3`.

2.3 Fonctions, fonctions récursives

Il y a *une seule construction* pour les fonctions en fouine, qui correspond à `fun x -> e` en Caml. Insistons : il ne fait pas sens d'avoir un traitement spécifique pour les "fonctions à plusieurs arguments". En fouine, tout se ramène à des fonctions à un seul argument. De même, il n'y a pas de construction spécifique pour l'application n-aire, car il n'y a que l'application binaire : `f 4 5`, c'est `(f 4) 5`.

Référez-vous aux transparents du premier cours pour vous remémorer la notion de clôture, qui est nécessaire à l'exécution de code fouine avec des fonctions.

Évaluer des fonctions. Une fonction est une valeur : en première approche, on traite `let f = fun x -> x+1 in f 3` comme `let x = 32 in x*52`. Pour associer une valeur à une fonction, on est amené à manipuler la notion de *clôture*. Considérons le code fouine suivant :

```
let k = 3 in
let f = fun x -> k*x in
let k = 12 in
prInt (f 5)
```

Cela affiche `15`. Au moment de l'appel à `f 5`, c'est la valeur de `k` au moment où `f` a été définie qui importe. La valeur correspondant à `f` est constituée du paramètre (`x`), du corps de la fonction (`k*x`) et de l'environnement (qui explique que `k` vaut `3`). On appelle cela une *clôture*, car la valeur ainsi obtenue n'a pas de variable libre (puisqu'elle se promène avec l'environnement qui donne un sens aux variables libres que contient le corps de la fonction).

Regardons maintenant

```
let k = 3 in
let g = fun x y -> k*x*y in
let f = g 4 in
let k = 12 in
prInt (f 5)
```

Cela affiche 60. Comprenez quelle est la valeur associée à `g`, et comment est calculée la valeur associée à `f` (spoiler : on ne remplace pas dans le corps de `g`).

On peut en principe optimiser, car en général, seule une partie de l'environnement n'est nécessaire pour clore. Par exemple, si dans les exemples précédents on avait aussi défini deux variables `a` et `b` avant de faire `k = 3`, les valeurs associées à `a` et `b` sont inutiles. Vous pouvez ignorer cette optimisation ; si vous ne l'ignorez pas, soyez conscient(e) du fait qu'il faut clore transitivement.

Formes d'écriture pour les fonctions. On peut déclarer une fonction avec `let f x z = ..`, ou avec `let f = fun x -> fun z -> ..`, ou avec `let f = fun x z -> ..`, ou un mélange de ces approches. C'est plutôt dans le parser qu'il faut gérer toutes ces possibilités d'écriture, puisque, répétons-le, il n'y a qu'une construction pour les fonctions.

On peut utiliser une fonction directement sans la nommer ("fonctions anonymes"), comme dans `let t = (fun x -> x*x) a in ...`

On peut appliquer les fonctions comme dans `f (g z) s t`.

L'analyse syntaxique de l'application (qui est notée simplement avec un espace en Caml) est un point délicat : cf. la partie 3.2.

Traitez aussi la difficulté en lien avec le moins unaire (`3 -1` c'est une expression arithmétique, pas l'application de `3` à `-1`).

Étapes. Ajoutez d'abord une unique forme de fonction, à savoir `fun x -> ...` Ayez une fouine qui marche avant de vous attaquer aux autres écritures possibles pour les fonctions. Voir aussi la partie 3.2 pour des étapes dans la mise en place de l'analyse syntaxique pour les applications.

2.3.1 Fonctions récursives

Construction `let rec`, en deux mots, pour définir des fonctions récursives (pas de message d'erreur si la fonction n'est pas récursive).

On ne s'intéressera qu'à des définitions récursives de *fonctions* (en Caml, on peut aussi définir d'autres choses récursivement, comme dans `let rec ones = 1::ones`).

Diverses écritures pour les fonctions récursives seront acceptées : `let rec f = fun x ->..`, et aussi `let rec f x y z = ..`, ou encore `let rec f = fun x y z -> ..`

Étapes. Intéressez-vous d'abord aux définitions récursives de la forme `let rec f = fun x -> ...` Ensuite, vous traiterez d'autres cas (un exemple particulier vous est fourni dans la batterie de tests).

2.4 Références et aspects impératifs

Ajouter les constructions `ref := ! () ;`

On peut stocker n'importe quelle valeur (entier, booléen, `()`, fonction, référence) dans une référence.

Représentation de la mémoire.
Important : Interdit d'implémenter les références à l'aide des références de Caml : on se donnera un grand tableau de taille arbitraire (par exemple 1234), qui correspondra à une représentation du tas.

Pas d'approche astucieuse pour l'allocation : lorsque le tas déborde, une exception est levée et fouine s'arrête *de manière gracieuse*, en indiquant qu'il n'y a plus de place.

Ce que l'on peut écrire en présence de références et de `unit`.

On peut écrire des fonctions prenant `()` en argument, comme par exemple dans `fun () -> prInt x`. Traiter cette particularité vous poussera à étendre la grammaire de *"ce qu'il est possible de mettre entre le `fun` et le `->`"*. Il en sera d'ailleurs de même pour *"ce qu'il est possible de mettre entre le `let` et le `=`"*. Cette réflexion est un avant-goût de la partie 2.6.

La construction ; qui sert à enchaîner deux expressions de manière séquentielle est reconnue, mais ne fait pas partie des expressions, au sens où le parser la traduit en un `let.. in`. Contrairement à Caml, on n'engendre pas de message d'avertissement (*warning*) lorsqu'on fait par exemple `3;4`.

Dans un premier temps, on pourra se contenter d'utilisations élémentaires de `ref`, qui ne sera pas traitée comme une "vraie fonction"[2]. À terme, il serait bon que `prInt` et `ref` soient vues comme des fonctions et puissent être passées en argument à des fonctions.

2.5 Exceptions

On étend fouine avec une seule exception, appelée `E`, et qui prend un `int` en argument.

```
exception E of int
```

(la ligne ci-dessus n'est pas à écrire dans vos programmes fouine ; il faut l'inclure en revanche si on veut exécuter un programme fouine à l'aide de Caml).

2.5.1 Comment ça marche en Caml

Les exceptions servent à ... gérer les cas exceptionnels.

Soit la fonction suivante, qui multiplie tous les entiers d'une liste.

```
let rec l_mul_naive = fun l ->
  match l with
  | [] -> 1
  | x::xs -> x*l_mul_naive xs
```

En faisant

```
let l = [ 2;4;1;0;3;5 ]

let k1 = l_mul_naive l
```

ça renverra `0` pour `k1`. Lors du calcul de `k1`, Caml considère la forme de la liste `l`, puis
s'appelle récursivement sur `[4;1;0;3;5]` en se souvenant qu'il faudra multiplier le résultat par 2, puis
s'appelle récursivement sur `[1;0;3;5]` en se souvenant qu'il faudra multiplier le résultat par 4, puis
s'appelle récursivement sur `[0;3;5]` en se souvenant qu'il faudra multiplier le résultat par 1, puis
s'appelle récursivement sur `[3;5]` en se souvenant qu'il faudra multiplier le résultat par 0, puis
s'appelle récursivement sur `[5]` en se souvenant qu'il faudra multiplier le résultat par 3, puis
s'appelle récursivement sur `[]` en se souvenant qu'il faudra multiplier le résultat par 5, puis
renvoie 1, puis
renvoie 5, puis
renvoie 15, puis
renvoie 0, puis
renvoie 0, puis
renvoie 0.

Vous le savez sans doute, les appels aux fonctions sont gérés à l'aide d'une pile, ce qui correspond au fonctionnement ci-dessus : un appel d'une fonction fait un "push", une fonction qui renvoie un résultat fait un "pop".

[2] Alors qu'en Caml :

```
# ref;;
- :  'a -> 'a ref = <fun>
```

Dans l'exécution ci-dessus, les parties colorées en violet et en orange sont désolantes.

Une première amélioration peut être apportée ainsi :

```
let rec l_mul_if = fun l ->
  match l with
  | [] -> 1
  | x::xs -> if x=0 then 0 else x*l_mul_if xs
```

Le calcul de `k2 = l_mul_if l` se déroule ainsi : Caml

s'appelle récursivement sur `[4;1;0;3;5]` en se souvenant qu'il faudra multiplier le résultat par 2, puis
s'appelle récursivement sur `[1;0;3;5]` en se souvenant qu'il faudra multiplier le résultat par 4, puis
s'appelle récursivement sur `[0;3;5]` en se souvenant qu'il faudra multiplier le résultat par 1, puis
renvoie 0, puis
renvoie 0, puis
renvoie 0.

On est content, car le `if then else` permet d'éviter les appels récursifs inutiles après avoir rencontré `0`. Voici une troisième version de cette fonction, pour être encore moins désolés :

```
let l_mul_exc = fun linit ->
  let rec lme_aux = fun l ->
    match l with
    | [] -> 1
    | x::xs -> if x=0 then raise (E 5420) (* entier arbitraire *)
                      else x*lme_aux xs
  in
  try lme_aux linit
  with (E n) -> 0
```

La fonction interne `lme_aux` fait les appels récursifs que faisait `l_mul_naive`. Cette fonction interne est appelée dans

```
  try lme_aux linit
  with (E n) -> 0
```

On voit ici que `lme_aux` est appelée avec `linit`, la "liste initiale", à savoir l'argument de `l_mul_exc`. Cet appel est "surveillé" par l'intermédiaire de la construction `try.. with`.

Au moment où on rencontre le premier `0` dans la liste, le `if` a pour effet de *lever une exception*, en faisant `raise (E 5420)`. À ce moment-là, tous les appels en attente sont court-circuités, et on remonte d'un coup jusqu'au `try.. with`. L'effet de `with (E n) -> ..` est de *rattraper l'exception*. Ici, on rattrape l'exception, et on renvoie directement zéro.

Intuitivement, tout se passe comme si on passait directement, en une étape, de l'état

`2 * (4 *` ★`)`

(★ désigne l'endroit où a lieu le calcul) à l'état où l'on a renvoyé `0`. On est content, car l'exception permet de court-circuiter le retour aux appels récursifs en attente (représentés ci-dessus par `2 * (4 *`) pour renvoyer directement `0`, sans calculer `4*0` puis `2*0`.

On peut remarquer que dans le cas présent, l'entier qui est passé avec l'exception est inutile. Dans d'autres situations, ça peut être un entier utile à un calcul, ou bien un code correspondant à un type d'erreur, ou autre.

Si un calcul "protégé" par un `try.. with` ne provoque pas d'exception, alors le résultat est renvoyé directement, et la branche `with` n'est pas utilisée. Ce sera le cas si on calcule `l_mul_exc [2;4;6]`, par exemple.

Si une exception n'est pas rattrapée par un `try.. with`, comme par exemple dans

```
let f x = raise (E (x+1)) in f 3
```

on "se prend l'exception dans la figure". C'est un usage assez courant des exceptions, qui correspondent à des siuations catastrophiques (`failwith` est souvent employé ainsi).

2.5.2 Comment traiter les exceptions en fouine

On se restreint à une seule exception, `E of int`, qui n'est pas déclarée dans les programmes fouine : on peut directement écrire `raise (E 3)`, mais pour tester les programmes avec Caml il faudra ajouter `exception E of int`, à l'image de ce qui est fait pour que Caml connaisse `prInt`.

À l'image de ce qui est demandé pour les références, on n'a pas le droit de s'appuyer sur les exceptions de Caml pour exécuter les exceptions fouine.

Version facile ("la monade maybe").

La première approche que vous pouvez envisager pour exécuter un programme fouine comportant des exceptions est d'étendre le type des valeurs. Cette extension peut être faite de manière plus ou moins élégante, elle peut en particulier se propager de manière pas très propre à tout le code.

Ce faisant, on "mime" les exceptions, mais le comportement d'un programme qu'on exécute avec fouine correspondra à la description donnée plus haut, avec la partie orange désolante. Mais bon, c'est une solution qui marche.

Version avec les continuations.

Si vous vous sentez plus ambitieux.se, et plus à l'aise, vous pouvez utiliser l'idée des continuations pour exécuter les exceptions en fouine (Section 5.2). Attention : il ne s'agit pas de traduire un programme fouine avec exceptions vers un programme sans exceptions, pour ensuite l'exécuter avec fouine, mais bien d'ajouter des continuations à l'interprète fouine, dont le rôle est de gérer les exceptions.

2.6 Couples, motifs, tuples

On étend fouine avec des couples. On peut écrire par exemple `let c = 1,2 in...` Plus important, on peut aussi écrire `let x,y = t in..`, ce qui permet de *déconstruire* un couple.

Motifs. Vous allez donc être amené.es à reprendre la structure des expressions, en introduisant la notion de *motif*. Dans `let x,y = t in..`, le motif `x,y` sert à reconnaître les couples, la première composante étant désignée par `x`, la seconde par `y`.

Réfléchissez à la notion de motif : le "motif élémentaire" c'est la variable, mais il y a aussi `_`, `()`, etc. Lorsqu'un motif rencontre une valeur, il "fait son marché", en découvrant la valeur associée à ses variables : on appelle cela le *filtrage*.

Ainsi, en ajoutant la notion de motif, vous introduisez le filtrage. Rendez-vous compte du fait que le filtrage est à l'œuvre dans les déclarations `let m = e1 in e2`, mais aussi dans les constructions pour les fonctions.

Étapes. Vous pouvez commencer par faire une version de fouine où l'on impose l'utilisation des parenthèses autour des couples. Une fois que ça marche, attachez-vous à éliminer l'usage des parenthèses.

Tuples. En Caml, les tuples sont primitifs : `(1,2,3)`, ça n'est ni `1,(2,3)`, ni `(1,2),3`, et on observe

```
# fst (1,2,3);;
Error: This expression has type 'a * 'b * 'c
       but an expression was expected of type 'd * 'e
```

On vous demande d'ajouter les tuples, et vous pouvez pour ce faire vous écarter de Caml, en décrétant que les tuples sont une notation pour des couples de couples (et l'évaluation ci-dessus n'engendrera pas d'erreur en fouine). Expliquez l'approche que vous avez adoptée dans votre fichier README.

2.7 Listes

Ajoutez `[]` et l'opérateur `::`, ainsi que le filtrage sur les listes, à l'aide des constructions `match.. with` et `function`. Pensez à adapter la notion de motif à la manipulation des listes : cf. ci-dessus, à propos des couples.

Ajoutez ensuite la possibilité d'écrire des listes en extension, comme dans `[3;2;4]`. Voir à ce propos la partie 3.3, qui donne des indications pour traiter la double signification du point virgule (comme séparateur de liste et comme opérateur de séquence).

Filtrage. Enrichissez la grammaire des motifs, et ajoutez la construction `match.. with`, qui permet de faire du filtrage.

Ajoutez également le mot-clefs `function`, afin de pouvoir définir une fonction par filtrage sur son argument, comme dans `let f = function [] -> 0 | x::xs -> 1` : ce code n'est pas accepté si on remplace `function` par `fun`. Vous pouvez traduire au niveau du parser un `function` en un `match with`, de manière à limiter le nombre de cas dans `expr` ; tant pis dans ce cas si le code affiché diffèrera du code entré, du moment que ce qui est affiché est équivalent, et est également accepté par Caml.

2.8 Types algébriques

Cette partie n'est à traiter que si vous avez traité les autres parties de manière exhaustive, et que tout marche. Vous pouvez ajouter les types définis par l'utilisateur. Les définitions de types précèdent l'unique expression qui est évaluée. On peut faire par exemple

```
type expr = Cst of int | Add of expr * expr | Mul of expr * expr
```

et puis on peut faire du filtrage sur les objets de `expr`. On est presque prêts alors à coder fouine en fouine !

L'ajout des types algébriques passe par l'ajout de la notion de constructeur, dont l'identifiant commence par une majuscule. Un constructeur prend en argument un tuple d'expressions, ou alors rien. Une valeur peut être un constructeur appliqué à un tuples de valeurs. On ne tient pas compte du typage, à part la vérification de l'arité : on rejette une application d'un constructeur à un tuple de la mauvaise arité (en particulier un constructeur tout seul "n'existe pas", sauf s'il est prévu qu'il n'ait aucun argument).

3 Indications pour les analyses lexicale et syntaxique

Les points d'entrée pour écrire vos analyseurs lexical et syntaxique sont :

- les transparents de cours, présentant les idées générales ;
- le fichier `README-parser.md` qui est inclus dans l'archive de départ pour le rendu 0.

Pour étendre le langage accepté en entrée de fouine avec de nouvelles constructions, il faut

1. étendre le type des expressions, et, le cas échéant, le type des valeurs

2. ajouter les léxemes correspondants dans `lexer.mll`, avec les expressions régulières correspondantes
3. ajouter les règles de grammaire dans `parser.mly`, en gérant l'interaction avec les autres règles de grammaire, qui peut être source d'ambiguïtés.

Erreurs à la compilation : ambiguïtés. La compilation des analyseurs lexical et syntaxique peuvent engendrer des conflits, appelés *shift/reduce* ou *reduce/reduce*. Ceux-ci correspondent à des situations d'ambiguïté, où l'automate ne sait pas choisir entre empiler ou réduire, ou entre réduire ou réduire. "Empiler" signifie avancer vers la droite dans la reconnaissance de la séquence de léxemes. "Réduire" signifie appliquer une règle de la grammaire pour reconnaître une suite de léxèmes.

Important : vous devez rendre du code qui compile, ce qui signifie en particulier sans conflit S/R ou R/R : vous devez rendre à chaque fois du code qui compile, et qui n'engendre pas de telles erreurs.

Écrire un analyseur syntaxique, c'est programmer, bien que l'approche soit inhabituelle si on pense à des langages de programmation usuels. Il s'agit de réfléchir avant de coder, de structurer le code (les règles de grammaire, en l'occurence) de manière raisonnée, de factoriser le code, de le tester.

Tout comme pour des programmes au sens habituel, il est primordial de tester *au fur et à mesure que l'on développe* : n'attendez pas d'avoir un gros fichier qui engendre une quantité industrielle de conflits avant de commencer à corriger.

3.1 Corriger des bugs dans le parser

On fait ici quelques remarques sur le travail sur le parser. Référez-vous au fichier README qui accompagne le rendu 0 pour davantage d'indications.

On n'insistera jamais assez sur le point suivant : pour corriger des conflits (shift/reduce ou reduce/reduce), commencez par réduire le problème, en travaillant sur un parser qui n'engendre pas 34985 conflits. Pour ce faire, désactivez des parties de la grammaire, et simplifiez les règles, afin d'isoler le problème.

Conflits shift/reduce. Un conflt Shift/Reduce intervient entre un lexème, qui veut être *shifté* (poussé sur la pile) et une règle qui veut être *réduite*.

Pour le résoudre, on commence par s'assurer du fait que le lexème a une priorité. On peut ensuite, si nécessaire, introduire une priorité factice qu'on associera à la règle souhaitant être réduite : plus faible que la priorité du lexème si on veut favoriser le shift.

Ça ne se résoud pas en donnant 1 token fictif à chaque regle du conflit, et en disant "c'est celle-ci qui gagne", car on a intuitivement un conflit entre *une* règle et un lexème.

Priorités : left, right, nonassoc. Autre point, les directives `nonassoc`, `left` et `right` sont gérées ainsi : lorsqu'il y a la même priorité, on résoud à l'aide de `left` ou `right` (si on a mis `nonassoc`, ça ne résoud pas).

C'est ce qui se passe quand on parse `3+4+2`, bien sûr. Ce mécanisme peut aussi être utilisé pour résoudre le conflit shift/reduce avec `if-then` vs `if-then-else`, en mettant `then` et `else` au même niveau, et en associant à droite ou à gauche (suivant le comportement que l'on souhaite obtenir).

3.2 Analyse des applications

Pour analyser $3 + 4 + 5$, c'est facile, on fait une règle pour la somme, et on dit que la somme est associative à gauche (par exemple) à l'aide de la directive spécifique `%left` de `menhir`.

La difficulté avec l'opérateur d'application c'est qu'il s'écrit sans symbole, en juxtaposant simplement l'expression en position de fonction et l'argument.

L'application est associative à gauche, et il faut dire cela au parser en structurant la grammaire de manière appropriée.

Le point de départ est la constatation du fait que si on écrit une règle comme

```
expression:
| expression expression { App($1,$2) }
```

on engendre des ambiguïtés.

Il faut en quelque sorte "programmer" la récursion à gauche de la règle pour l'application.

Lorsqu'on analyse `f x (g t) z`, la bonne approche est d'isoler `z` comme l'argument de l'application qu'on est en train d'analyser, `f x (g t)` étant l'expression en position de fonction. On continue ensuite récursivement : `(g t)` est l'argument de `f x`, et enfin `x` est l'argument de `f`.

Ainsi, on peut introduire un non terminal pour représenter les applications, avec une règle de la forme :

```
applic:
| applic sexpr { App($1,$2) }
```

L'idée est que `sexpr` est un non-terminal couvrant tout ce qui peut être reconnu comme argument d'une fonction "tel quel" : c'est le cas des identificateurs et des entiers (d'où les règles évidentes correspondantes pour `sexpr`).

Pour passer une expression qui n'est pas un `sexpr` en argument d'une application, il faut nécessairement parenthéser, comme dans `(g t)` ci-dessus. Il y a donc une règle de la forme

```
sexpr:
| LPAREN expression RPAREN { $2 }
```

L'enjeu est donc de répartir dans la grammaire ainsi "étagée" les différentes constructions du langage, en se fondant si nécessaire sur le comportement de Caml.

Étapes. Commencez par reconnaître uniquement les suites d'identifiants comme `f x y z t`, en s'assurant du fait que l'expression est bien reconnue. Pour ce faire, on peut "court-circuiter" dans `parser.mly` la grammaire existante en introduisant un non-terminal supplémentaire, et court-circuiter dans `main.ml` l'évaluation (qui engendrerait une erreur, puisqu'on n'a pas les déclarations), en lançant uniquement une fonction d'affichage.

3.3 Séquence et listes.

Si on a une grammaire comportant à une règle

```
  expression: expression SCOLON expression     { ... }
```

et des règles

```
  expression: LBRACK exprlist RBRACK           { $1 }
(...)
  exprlist: expression SCOLON expression       { ... }
```

il y a une ambiguïté liée à l'usage du point virgule (`SCOLON`) : par exemple, on ne sait pas si la liste `[2;4]` est une liste à deux éléments, ou une liste à un seul élément, qui serait alors une séquence.

Pour dire qu'il s'agit d'une liste à deux éléments, il faut pouvoir parler des expressions sans séquence. Une manière de faire cela est d'introduire les séquences d'expressions. Intuitivement, cette notion est "tout en haut" dans la grammaire :

```
  top_expression: exprseq                          { $1 }

  exprseq:
      expression                                   {$1}
    | exprseq SCOLON exprseq                       { ... }
;
```

À noter que l'on s'appuie sur

```
%left SCOLON            /* on pourrait mettre %right */
```

pour gérer l'associativité de `SCOLON` dans la séquence.

Dans les listes, on garde des `expression` et pas des `exprseq`, ce qui correspond à dire que "`;`" est le separateur de listes et pas la séquence.

Un point important, pour pouvoir reconnaître par exemple la liste à deux éléments `[2;(3;4)]`, est de mettre `exprseq` lorsqu'on insère des parenthèses :

```
expression: LPAREN exprseq RPAREN            { $2 }
```

Autrement dit, dès qu'on met des parenthèses, on "repart tout en haut" dans la grammaire.

Dans le même ordre d'idées, il s'agit de comprendre où est-ce que l'on met des `exprseq`, et où est-ce que l'on met des `expression`. Par exemple, il faut transformer

```
  | LET VAR EQUAL expression IN expression { LetIn($2,$4,$6) }
```

en

```
  | LET VAR EQUAL expression IN exprseq { LetIn($2,$4,$6) }
```

Pour s'en convaincre, on peut observer ceci :

```
# let x = 3 in x;x;;
Warning 10: this expression should have type unit.
- : int = 3
```

On voit que `x` est bien "defini deux fois" dans `x;x`, ce qui indique la priorité.

En faisant cette transformation, on récupère alors un conflit *shift/reduce* :

```
10: shift/reduce conflict (shift 19, reduce 3) on PLUS
state 10
        exprseq : expression .  (3)
        arith : expression . PLUS expression  (8)
```

L'ambiguité provient par exemple de `3;4+5` : on reconnaît `3` comme `expression`, puis comme `exprseq`. Ensuite on reconnaît `4` comme `expression`, et à ce moment-là on ne sait pas si l'on doit "promouvoir" `4` en `exprseq`, ou bien rester dans les `expression` pour empiler le `+`.

Pour implémenter le bon comportement (empiler le `+`), on dit explicitement que la règle qui est candidate pour faire un *reduce* qu'elle est de priorité moindre.

Pour ce faire, on introduit une priorité factice :

```
%token UNDERPLUS
```

puis

```
%nonassoc UNDERPLUS
%left PLUS
```

et on reprend la grammaire pour `exprseq` :

```
  exprseq:
    expression %prec UNDERPLUS              {$1}
    | exprseq SCOLON exprseq                { ... }
;
```

4 Ajouter du typage — inférence de types

Il s'agit d'ajouter l'inférence de types à votre fouine : on commence par vérifier si le programme est typable. Si c'est le cas, on l'exécute. Sinon, on proteste et on s'arrête.

4.1 Inférence de types monomorphe

Fonctions (et entiers et booléens).

Vous avez vu en cours de Théorie de la Programmation comment fonctionne l'inférence de types en Fun. Le but est d'adapter et d'étendre cette approche à fouine, en commençant par le cœur du langage fouine : entiers, booléens et fonctions.

L'inférence de types se fait en deux étapes :

1. on engendre des *contraintes de typage* en parcourant le programme que l'on analyse ;
2. ces contraintes constituent un *problème d'unification*, que l'on résoud.

Mise en place. Pour mettre en œuvre l'inférence de types, il vous faudra :

- Définir ce que sont les types, et les contraintes de types. Replongez-vous dans le cours de théorie de la programmation, vous y verrez en particulier que les types contiennent des variables de types (notées $\alpha, \beta, \gamma, \dots$), qui sont nécessaires pour exprimer les contraintes de types.
- Mettre en place les divers fichiers (sans surprise, un pour l'algorithme d'unification, l'autre pour l'inférence).

  Le code devrait être organisé de telle manière que cela soit transparent vis-à-vis de l'approche utilisée pour implémenter l'algorithme d'unification (naïve ou sophistiquée).

Ce travail de mise en place est à faire à deux, il permettra de vous répartir l'implémentation de l'inférence de types.

Programmer l'unification. Vous pouvez exploiter le TP du cours de théorie de la programmation qui traite de l'unification, et le cours qui va avec.

Si l'unification échoue, on arrête tout.

Si l'unification réussit, le programme est typable. Il faut avoir prévu ce qu'il faut pour pouvoir récupérer le type de l'expression qui a été passée à fouine, ainsi que de ses sous-expressions — tout du moins *certaines* de ces sous-expressions, cf. l'option `-showtypes` ci-dessous.

Comportement, options. Sans options, `fouine` type le programme, n'affiche rien si le programme est typable, et l'exécute.

Si le programme n'est pas typable, affichez un message d'erreur de votre cru, avec des indications sur l'origine du problème.

Programmez aussi les options suivantes :

`-notypes` : désactive l'inférence de types, et interprète directement les programmes.

`-showtypes` : affiche le type inféré pour toutes les déclarations en surface, l'expression principale étant désignée par `-`. Ainsi, sur le programme

```
let f x = let y = x+1 in y in
let a = 12 in
  f a
```

`fouine -showtypes` répondra

```
f :  int -> int
a :  int
- :  int
```

Les types contenant des inconnues de types / des variables. Si on lance l'inférence sur `fun x y -> x`, il n'y a pas de problème, c'est typable.

Que faire au moment d'afficher le type ? Vous pouvez par exemple afficher quelque chose qui ressemble à ce qu'afficherait Caml (`'a -> 'b -> 'a`). Ou alors afficher quelque chose comme `T0 -> T1 -> T0`.

Il est bon de garder à l'esprit que les `'a`, `'b` de Caml correspondent à un système *polymorphe*, abordé à la partie 4.2, alors que le système que vous implémentez dans cette partie est monomorphe. Vous pouvez vous en convaincre en contemplant le programme

```
let id x = x in fun f -> f (id 3) (id false)
```

et en comprenant pourquoi vous ne savez pas le typer avec le système de types monomorphe.

Extensions.

Une fois que l'inférence a été testée pour les fonctions, traitez les extensions suivantes, dans cet ordre :

- Références : ajout de `ref` sur les types, et de `unit`.
- Tuples : ajout de `*` sur les types (si votre fouine a les tuples)
- Exceptions : a priori il n'y a rien à ajouter dans le langage des types, mais il faut pouvoir typer des programmes qui utilisent `raise` et `try.. with`.

  ajout du type `exn`, si vous traitez les exceptions comme des valeurs (cf. la différence entre les fonctions `let f k = raise (E k)` et `let f k = E k`).

4.2 Le polymorphisme

On vous demande ici d'adapter votre code pour traiter le polymorphisme (les types avec des `'a` en Caml).

Options.
Important : on veut que vous ayez les deux versions de l'inférence de types ; une option `-monotypes` permettra d'exécuter l'inférence de types monomorphes (sans option, c'est la version polymorphe qui tournera). L'option `-monotypes` s'ajoute donc aux options `-notypes` et `-showtypes` qui sont en lien avec l'inférence de types.

À vous de réfléchir afin de combiner les deux versions de l'inférence de types sans trop dupliquer le code.

Pour prendre en compte le polymorphisme, les "briques de base conceptuelles" sont les mêmes, mais elles ne sont pas agencées de la même manière.

Schémas de types. Outre les types, on manipule aussi des **schémas de types**.

$$\tau \quad ::= \quad \texttt{int} \mid \texttt{bool} \mid \tau_1 \to \tau_2 \mid \alpha \qquad\qquad \rho \quad ::= \quad \tau \mid \forall \alpha.\rho$$

Les ρ désignent les schémas de types (les α ci-dessus sont les inconnues de types de l'inférence de types telle que vue en cours de théorie de la programmation : c'est juste une question de notations).

Notez qu'on a une grammaire à deux étages : on n'ajoute pas directement la quantification sur les types dans la grammaire des types. Ainsi,

$$\forall \alpha.\alpha \to (\forall \beta.((\beta \to \beta) \to \texttt{int})) \to \alpha$$

n'est pas un schéma de types, car la quantification sur β est sous une flèche.

Important : on vous demande d'implémenter les schémas de types, il ne faut pas essayer d'adapter ce qui est expliqué sur l'inférence de types polymorphes en maintenant la structure des types qui est utilisée pour le typage monomorphe.

Généralisation, spécialisation. La quantification universelle permet de généraliser un type en le transformant en un schéma de types. Intuitivement, cela permet de donner le type `'a -> 'a` à la fonction `fun x -> x`.

Il faut bien comprendre que tout se passe dans les `let`. En particulier, le type de `fun x -> x` n'est pas `'a -> 'a` : ce n'est pas à un `fun` que l'on généralise. En effet, *le polymorphisme n'est à l'œuvre que dans les déclarations*. Par exemple, dans `let f = fun x -> x in e`, le type associé à `f` est $\forall\alpha.\alpha \to \alpha$ pour typer `e`.

Par conséquent, quand on infère le type de `fun x -> e`, on fait comme dans le cas monomorphe. En revanche, pour inférer le type de `let x = e1 in e2`,

- on infère le type de `e1` : ça donne un type τ ;
- on **généralise** ce type : ça donne un schéma de types, mettons ρ, qui est associé à `x` pour inférer le type de `e2` ;
- l'environnement de typage qui sert pour inférer le type de `e2` est donc une structure associant *un schéma de types* à chaque identifiant de variable ;
- au moment de typer un usage de `x` dans `e2`, on **spécialise** le schéma ρ, qui est instancié avec de nouvelles variables de types. On prend pour ce faire de nouvelles variables de types à chaque fois : c'est ici qu'on met en œuvre le polymorphisme.

Ce mécanisme de généralisation puis spécialisation permet d'associer une instanciation différente à chaque usage de `x` dans `e2` : c'est le polymorphisme, au sens où on peut utiliser `x` selon des types différents, mais tous compatibles au sens où ils proviennent d'un même schéma de types.

Algorithme d'unification : ne pas le changer, mais l'appeler plus souvent. Au vu de ce qui est décrit ci-dessus, on peut observer que :

- l'unification est utilisée, comme précédemment, sur les types, et pas sur les schémas de types ;
- alors que dans le cas monomorphe on procède en deux phases globales (d'abord engendrer le problème d'unification, puis le résoudre), en présence de polymorphisme, on résoud des problèmes d'inférence au fur et à mesure qu'on parcourt le programme : à chaque `let x = e1 in e2`, on infère le type de `e1` (ce qui fait intervenir un appel à l'unification) avant de passer à l'inférence pour `e2`.

Formellement. On donne ci-dessous les règles de typage (cf. ce qui a été dit en cours dans la partie "théorique" à propos de l'inférence de types en présence de polymorphisme).

Les Γ sont des ensembles de couples notés $x : \rho$.

On note $\tau_0[\widetilde{\tau}/\widetilde{\alpha}]$ le type obtenu en remplaçant les variables de types $\widetilde{\alpha}$ par les types $\widetilde{\tau}$; le résultat est un type.

$\widetilde{\alpha} \cap \Gamma = \emptyset$ signifie qu'aucune variable de type dans α n'apparaît pas libre dans Γ.

$$\frac{\Gamma \vdash e_1 : \texttt{int} \qquad \Gamma \vdash e_2 : \texttt{int}}{e_1 + e_2 \vdash \texttt{int}} \qquad \frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \texttt{fun}\ x \ \texttt{->}\ e : \tau_1 \to \tau_2} \qquad \frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2}$$

$$\frac{}{\Gamma \vdash x : \tau_0[\widetilde{\tau}/\widetilde{\alpha}]}\ \Gamma(x) = \forall\widetilde{\alpha}.\tau_0 \qquad \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \forall\widetilde{\alpha}.\tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}\ \widetilde{\alpha} \cap \Gamma = \emptyset$$

Rien de surprenant s'agissant des règles pour les fonctions ; notez toutefois que lorsqu'on écrit $\Gamma, x : \tau_1$ dans la règle pour typer une fonction, on ajoute l'hypothèse $x : \tau_1$ à Γ, où τ_1 est vu comme un schéma de types. De fait, seule la règle pour typer les déclarations permet d'ajouter des hypothèses de typage avec des types quantifiés.

Tout se passe dans les règles pour la déclaration (où l'on fait la généralisation) et pour la variable (où l'on fait la spécialisation).

Attention à la généralisation. Généraliser un type en un schéma de types signifie rajouter des $\forall$ pour quantifier sur les variables de types. On ne généralise cependant pas par rapport à toutes les variables, mais uniquement par rapport à celles qui n'apparaissent pas ailleurs dans l'environnement de typage.

Voici une illustration du risque :

$$\dfrac{\dfrac{\dfrac{}{x : \alpha \vdash x : \alpha} \qquad \dfrac{}{x : \alpha, y : \forall\alpha.\alpha \vdash y : \beta}}{x : \alpha \vdash \texttt{let}\ y = x\ \texttt{in}\ y : \beta}}{\emptyset \vdash \texttt{fun}\ x \longrightarrow \texttt{let}\ y = x\ \texttt{in}\ y : \alpha \to \beta}$$

Dans la dérivation ci-dessus, au moment d'appliquer la règle pour la déclaration, on généralise le type que l'on associe à y, alors que "α traîne encore". Cela permet de déduire le type $\alpha \to \beta$ pour ce qui est fondamentalement l'identité (pourquoi est-ce une catastrophe ?).

Cela se traduit par la condition $\widetilde{\alpha} \cap \Gamma = \emptyset$ pour contrôler la généralisation dans la règle de typage pour les déclarations.

Attention aux références (en présence de polymorphisme). Une autre limitation de la généralisation apparaît en lien avec le typage des références. Si on combine le polymorphisme et les références sans faire attention, on risque de typer le programme suivant :

```
let f = ref (fun z -> z) in
    begin
      f := fun x -> x+1;
      !f false
    end
```

La solution est de restreindre la généralisation dans le typage d'une déclaration :

- On ne généralise pas le type inféré pour un `ref`. Autrement dit, lors de l'inférence pour `let x = ref e in e'`, on infère un type τ pour `ref e`, et on passe à l'inférence pour `e'` en associant τ (vu comme schéma de types) à `x`.
- On ne généralise pas le type inféré pour une application : même chose que ci-dessus pour `let x = e1 e2 in e'`. La raison est qu'on a peur que l'évaluation de `e1 e2` ne fabrique des références, donc soit "risquée".

  Vous pouvez vérifier ce qui se passe en typant `(fun x -> x) (fun z -> z)` (l'identité *appliquée* à elle-même) dans Caml : on récupère un type monomorphe (signalé par un _).

Il existe des manières plus fines de restreindre la généralisation en présence de références, vous pouvez chercher cela sur internet.

Bonus : autoriser les types récursifs Cette partie n'est à traiter que si vous avez traité les autres parties de manière exhaustive, et que tout marche.

Elle se présente comme une partie "ouverte", à vous de voir comment procéder.

Il vous est demandé d'ajouter une option `-rectypes` à votre inférence de types. Cette option permet de manipuler des types récursifs, et ainsi accepter davantage de programmes.

Voici sur un exemple ce que ça donne en Caml :

```
ocaml -rectypes
OCaml version 5.2.1

# fun z -> z z;;
- : ('a -> 'b as 'a) -> 'b = <fun>
```

Un type récursif est représenté à l'aide d'équations entre types, le `as` tenant lieu ici de "=". L'idée générale est que l'unification échoue en cas de conflit entre deux constructeurs de types (un entier rencontre une fonction, par exemple), mais on gère les cas de boucle infinie.

5 Continuations

5.1 Programmation par continuations, principes

La *programmation par continuations* est un style de programmation où les fonctions ont toutes, intuitivement, un paramètre supplémentaire, qui est une fonction que l'on appelle pour renvoyer le résultat.

Ainsi, la définition ci-dessous de la fonction `f` à gauche devient, en style par continuation, la définition à droite :

```
let f = fun x -> x*x            let f = fun x -> fun k -> k (x*x)
                                ou, si l'on veut,   let f x k = k (x*x)
```

Ici `k` est la *continuation*. `k` est une fonction, qui représente le "futur du calcul" effectué par `f`. À titre d'intuition, alors que dans la version originale de `f` (à gauche), on renvoie le résultat de l'évaluation de `x*x`, comme s'il y avait un `return` implicite, avec les continuations, on appelle `k` en lui passant ce résultat.

On peut adopter systématiquement le style par continuations, ce qui donne, pour une fonction qui appelle une autre fonction :

```
let f x = x*(g x)            let f x k = g x (fun r -> k (x*r))
```

À droite, `f` passe directement la main à `g`, en lui fournissant la continuation `fun r -> k (x*r)`. Cette continuation se lit comme "lorsque `g` aura fini de calculer et renverra un résultat `r`, on multiplie par `x` et on envoie le résultat à `k`, la continuation qui avait été transmise lors de l'appel originel à `f`."

5.2 Implémenter les exceptions à l'aide de continuations

Vous pouvez prendre la fonction `eval` qui est le cœur de l'interprète fouine, et la réécrire en style par continuation.

En particulier, pour les appels récursifs, comme par exemple pour évaluer `e1 + e2`, on enchaîne les continuations, ce qui donne, à quelques détails près :

```
eval e2 env (fun n2 -> eval e1 env (fun n1 -> k (n1+n2)) )
```

où `k` est la continuation qui "attend" le résultat de l'évaluation de `e1+e2`.

Pour traiter les exceptions à l'aide de continuations, on travaille avec *deux continuations* : au lieu d'écrire `fun k ->`, on écrit `fun (k,kE) ->`. La continuation `k` est le futur du calcul, au sens usuel pour les continuations. `kE` est le "futur exceptionnel". Un calcul peut choisir entre répondre "tout va bien, et le résultat est `5`", ce qui se traduit par l'appel `k 5`, ou dire "il y a un problème, je lève une exception, que je décore avec l'entier `12`", ce qui se traduit par l'appel `kE 12`.

Dit autrement, la seconde continuation, `kE`, correspond au `try. .with` qui est en train de surveiller le calcul courant.

5.3 Traduire pour éliminer les exceptions

Une autre approche pour traiter les continuations consiste à programmer une traduction (ou *compilation*) de (fouine+exceptions) vers fouine.

On adopte l'approche décrite à la partie 5.2, pas pour interpréter les programmes, mais pour engendrer un programme sans exceptions (et, soit dit en passant, fort illisible).

On présente ici les grandes lignes de la traduction, à vous d'affiner et de trouver le reste[3]. La traduction de `e`, notée $[\![e]\!]$, se fait par induction sur l'expression `e`. On décrit ci-dessous un certain nombre de cas intervenant dans cette définition.

[3] Vous pouvez bien sûr nous contacter si vous avez des doutes.

- $[\![$`12`$]\!]$ = `fun k -> k 12`
  L'entier `12` est traduit en la fonction qui attend une continuation et lui envoie l'entier `12`.
- $[\![$`e1 + e2`$]\!]$ = `fun k ->` $[\![$`e2`$]\!]$ `(fun v2 ->`
  $[\![$`e1`$]\!]$ `(fun v1 -> let v = v1+v2 in k v) )`
  Pour évaluer `e1+e2`, on évalue `e2`, en lui passant une continuation qui évaluera `e1`, puis renverra le résultat final à `k`, la continuation originelle.
  Le code ci-dessus peut aussi être écrit ainsi :

```
[[e1 + e2]] = fun k ->
  let k2 = fun v2 ->
    let k1 = fun v1 ->
      let v = v1+v2 in k v in
    [[e1]] k1 in
  [[e2]] k2
```

- $[\![$`let x = e1 in e2`$]\!]$ = `fun k -> (` $[\![$`e1`$]\!]$ `(fun x ->` $[\![$`e2`$]\!]$ `k) )`
  On évalue `e1`, puis `e2`.
  On est astucieux en gardant le nom de variable `x` dans la version traduite. Ainsi, le résultat de l'évaluation de `e1` remplacera bien `x` dans `e2` au moment de passer à l'évaluation de `e2`.
- $[\![$`x`$]\!]$ = `fun k -> k x`
  La variable est traitée comme une valeur : on la passe directement à `k`. On peut remarquer que c'est cohérent, puisqu'une variable sera toujours substituée par une valeur.
- $[\![$`(fun x -> e)`$]\!]$ = `fun k -> k (fun x ->` $[\![$`e`$]\!]$`)`
  C'est le même principe : comme une fonction est une valeur, on renvoie la valeur à `k`, sauf qu'il faut traduire cette valeur avant de l'envoyer (d'où le $[\![$`e`$]\!]$).
- $[\![$`e1 e2`$]\!]$ = `fun k -> . . .`
  Pour l'application, c'est "presque" comme pour une addition : on évalue `e2`, puis `e1`, simplement on n'additionne pas les deux valeurs pour passer le résultat à `k`. À la place, on déclenche un calcul, en appelant une fonction avec son argument et sa continuation.
  Pour vous assurer que vous avez bien compris, relisez la traduction des fonctions, et vérifiez que "ça colle" lorsqu'on écrit `v1 v2 k`.

- **Gestion des exceptions.**
  Ce qui précède concerne la traduction par continuations du langage fonctionnel, sans exceptions.
  Afin de pouvoir traiter les exceptions, on doit en réalité adopter une transformation par continuations où l'on passe *un couple* de continuations, et non pas une seule continuation. Ce couple s'écrit typiquement `(k,kE)`, où `k` est la continuation normale, comme dans ce qui précède, et `kE` est la continuation exceptionnelle, que l'on invoque lorsque l'on veut lancer une exception.
  Ainsi, l'opération `raise` se traduit de la manière suivante :
  $[\![$`raise (E e)`$]\!]$ = `fun (k,kE) -> . . .`
  L'idée bien sûr est qu'il n'y ait pas de `raise` dans le programme résultant de la traduction : on traduit vers fouine sans exceptions.

  On vous laisse compléter la définition suivante :
  $[\![$`try e1 with (E n) -> e2`$]\!]$ = `fun (k,kE) -> . . .`

Définitions récursives. Pour traduire les fonctions récursives, vous pouvez vous inspirer de la *récursion ouverte*.

```
# let fact fact n = if n<=1 then 1 else n*(fact (n-1));;
val fact : (int -> int) -> int -> int = <fun>
# let rec factorielle = fact factorielle;;
Error: This kind of expression is not allowed as right-hand side of `let rec'
```

```
# let rec factorielle n = fact factorielle n;;
val factorielle : int -> int = <fun>
```

5.4 Programmer la traduction

Le code produit par la traduction par continuations (appelée CPS pour *continuation passing style*) est rapidement illisible.

Vous pouvez faire usage de déclarations, comme dans l'explication de la traduction de la somme ci-dessus, pour rendre les choses un peu plus lisibles.

S'agissant de l'implémentation, il serait bon de ne pas recopier directement la version papier de la traduction dans un fichier `.ml`, avec moult `Fun(..)` et `Pair(..)`. Essayez de définir des outils vous permettant d'écrire de manière lisible les morceaux de code traduit.

Optimisations : réductions administratives. L'option `-optim` implémente une version spécialisée de la traduction. L'idée est la suivante : dans certains cas, $[\![\texttt{e}]\!]$ est une expression qui peut se réduire vers une expression `e'`, car on peut déjà faire certaines réductions β, dans la mesure où l'on a un argument qui est une valeur. C'est le cas par exemple si l'un des deux arguments (ou les deux) d'une application est une variable.

Il s'agit donc de :

- trouver des cas de ce style, pour lesquels on peut définir une version optimisée de la traduction ;
- se donner la possibilité de raffiner la traduction pour n'appliquer la version "bateau" que lorsqu'on ne peut pas faire autrement, et appliquer la version optimisée lorsque c'est possible ;
- documenter les optimisations que vous avez implémentées dans le README (ou dans un fchier à part, mentionné dans le README, si vous tenez absolument à faire du LaTeX).

6 pieuvre, un petit assistant de preuve

6.1 Mise en place : λ-calcul, réduction, typage

On souhaite implémenter une version du λ-calcul simplement typé avec $*$, $+$, et les types constants 1 et 0. Comme vu dans la partie théorique du cours, cela correspond à la déduction naturelle pour la logique propositionnelle avec conjonction, disjonction, vérité et faux. On commence par se restreindre aux types flèche et au type 0 (qui représente le faux).

6.1.1 λ-calcul, β-réduction.

On commence par le λ-calcul, dans sa version non typée. Les λ-termes sont donnés par la grammaire

$$M, N \quad ::= \quad \lambda x\colon A.M \mid M\ N \mid x \mid \mathsf{exf}(M\colon A)$$

L'abstraction et la construction exf (qui correspond à la règle d'élimination du faux en déduction naturelle) comportent une annotation de type. On parle plus bas de typage, pour ce qui est de la β-réduction et de l'α-conversion, on ignore ces annotations. Ainsi $\lambda x : A.x$ est α-convertible à $\lambda z : A \to A.z$, par exemple.

Voici la syntaxe d'entrée correspondant à la grammaire ci-dessus.

```
fun (x:A) => M      M N      x      (* lambda-calcul *)
exf(M:A)     (* ex falso quodlibet, l'élimination du faux *)
```

(on met un "=>" dans `fun (x:A) => M` pour se rapprocher de la syntaxe de Rocq.)

Les noms de variables commencent par une suite non vide de lettres minuscules, et se terminent par une suite éventuellement vide de chiffres.

À noter que vous pourrez être amené.es à α-convertir les λ-termes, lorsque vous les réduirez, sans quoi votre implémentation de la β-réduction sera fausse. Il pourra être judicieux de garder trace des noms fournis lors de la saisie, pour que $\lambda u.M$ soit éventuellement α-converti en $\lambda u_{17}.M'$ plutot qu'en $\lambda x_{1927}.M''$, ceci afin de garder un minimum de lisibilité pour l'utilisatrice/teur.

λ-termes, α-conversion, β-réduction. On appellera `lam` le type représentant les λ-termes.

Implémentez les fonctions suivantes, et les options de pieuvre qui vont avec (cf. plus bas).

- `affiche_lam : lam -> unit` affiche un λ-terme avec la même syntaxe qu'en entrée
- `alpha : lam -> lam -> bool` teste si deux λ-termes sont α-convertibles

  Quand on arrive à un lieur, on instancie avec une variable fraîche des deux côtés.
- `betastep : lam -> lam option` fait un pas de β-réduction, et renvoie `None` si on a une forme normale.

  La β-réduction est fondée sur l'axiome suivant : $\dfrac{}{(\lambda x.M)\ N \quad \longrightarrow_\beta \quad M[N/x]}$

  Contrairement à ce qui est fait pour fouine, on implémente la relation qui réduit partout dans les λ-termes, y compris sous les λ et à l'intérieur des exf. Et on ne se promène pas avec un environnement, on substitue à l'intérieur du terme.

  Au moment de faire des β-réductions, on ne tient pas compte des annotations de types que contiennent les λ-termes (mais on ne les efface pas).

  Stratégie. On vous recommande de réduire systématiquement le redex situé le plus "en bas à gauche" dans le λ-terme que l'on doit réduire : on fonce sous les abstractions, et à gauche dans les applications. Ça n'est pas toujours la manière la plus efficace de réduire, mais on évite ainsi les divergences quand on peut (c'est un théorème en λ-calcul).
- `reduce : lam -> unit` qui affiche les réductions d'un λ-terme jusqu'à atteindre une forme normale, ou part en boucle infinie. Cette fonction s'appuiera sur les précédentes.

6.1.2 Types.

Dans un premier temps, on n'a que les types flèche et le faux : $A, B \quad ::= \quad X \mid A \Rightarrow B \mid \bot$

Et en syntaxe ascii :

```
A B A1 B1
->
False
~A (* uniquement pour le parser, qui remplace ``à la volée'' par A -> False:
            on ne veut pas gérer la tactique unfold *)
```

Les variables propositionnelles commencent par une séquence non vide de lettres majuscules, et se terminent par une séquence potentiellement vide de chiffres : `A B A1 BB21 LOUIS12`.

On définira les types suivants :
. `ty` : le type Caml représentant les types simples
. `gam` : le tye Caml représentant les environnements de typage

ainsi qu'une fonction

- `typecheck : gam -> lam -> ty -> bool` qui implémente la vérification de types. Cette fonction est notamment utilisée à la fin d'une preuve, pour vérifier qu'on a effectivement une preuve de la propriété annoncée.

  Les annotations de type aident pour écrire `typecheck`. La difficulté est que pour typer une application, il faut deviner le type de l'argument. Cependant, il est possible de reconstruire le type de toute expression à partir de l'environnement de type `gam` et des annotations de type. Votre fonction qui doit faire de la vérification de types sera de fait apparentée à une fonction qui fait de l'inférence de types, mais de l' *"inférence de types guidée"*, en quelque sorte. On vous conseille de vous inspirer de la partie 4.4.5 des **notes de cours de S. Mimram**, sur le typage bidirectionnel, pour implémenter cette partie.

6.2 Manipulation d'une preuve

Pour démarrer une preuve, on utilisera le mot-clef `Goal`, et on ne quantifiera pas sur les variables propositionnelles. Comme ceci :

```
Goal A -> (B -> A) -> A.
```

Voici un exemple de preuve interactive en Rocq[4] :

[4] Pour obtenir ce comportement en Rocq, sans avoir à quantifier sur les `A`, `B`, on peut faire précéder le code qui suit par
```
Section pieuvre.
Context {A B C D E F G : Prop}.
```

```
Rocq < Goal A -> (A -> A -> A) -> A.
1 goal

  ============================
  A -> (A -> A -> A) -> A

Unnamed_thm < intro.
1 goal

  H : A
  ============================
  (A -> A -> A) -> A

Unnamed_thm < intro.
1 goal

  H : A
  H0 : A -> A -> A
  ============================
  A

Unnamed_thm < apply H0.
2 goals

  H : A
  H0 : A -> A -> A
  ============================
  A

goal 2 is:
 A

nnamed_thm < exact H.
1 goal

  H : A
  H0 : A -> A -> A
  ============================
  A

Unnamed_thm < Show Proof.
(fun (A : Prop) (H : A) (H0 : A -> A -> A) => H0 H ?Goal)

Unnamed_thm < exact H.
No more goals.

Unnamed_thm < Qed.
```

et voici ce qui se passe du point de vue de la construction de la preuve dans deux étapes ci-dessus :

$$\dfrac{\overset{?}{}}{\underbrace{H : A, H0 : A \to A \to A}_{\Gamma} \vdash A}$$

--- apply H0 --->

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash A \to A \to A}\,Ax \qquad \dfrac{\overset{?}{}}{\Gamma \vdash A}}{\Gamma \vdash A \to A}\,\Rightarrow_E \qquad \dfrac{\overset{?}{}}{\Gamma \vdash A}}{\Gamma \vdash A}\,\Rightarrow_E$$

--- exact H --->

$$\dfrac{\dfrac{\dfrac{}{\Gamma \vdash A \to A \to A}\,Ax \qquad \dfrac{}{\Gamma \vdash A}\,Ax}{\Gamma \vdash A \to A}\,\Rightarrow_E \qquad \dfrac{\overset{?}{}}{\Gamma \vdash A}}{\Gamma \vdash A}\,\Rightarrow_E$$

À noter que dans l'extrait de preuve ci-dessus, on a omis d'indiquer les étapes correspondant aux applications de la tactique `intro`, qui correspondent à des usages de la règle $\Rightarrow_I$.

Le point de départ serait en réalité ainsi :

$$\dfrac{\dfrac{\dfrac{\overset{?}{}}{H : A, H0 : A \to A \to A \vdash A}}{H : A \vdash (A \to A \to A) \to A}\,\Rightarrow_I}{\vdash A \to (A \to A \to A) \to A}\,\Rightarrow_I$$

État de preuve. Une preuve en cours de développement est un λ-terme avec des trous. Ces trous sont en correspondance avec les buts courants, qui sont de la forme $\Gamma \vdash ? : A$. La preuve de départ se résume à un trou. Au fur et à mesure que des tactiques sont appliquées, la preuve prend forme, une tactique pouvant être vue comme une manière de transformer un trou en un terme, qui à son tour peut contenir zéro ou plusieurs trous.

Il convient de réfléchir à la représentation d'une preuve incomplète, d'un trou dans une preuve, et à la mise à jour d'une preuve lors de l'application de tactiques.

Comme en Rocq, on affiche le sous-but courant en énumérant les hypothèses (le Γ), puis en mettant la conclusion sous une barre `================`. Les sous-buts secondaires sont également affichés, mais partiellement, car on montre juste leur conclusion.

Appliquer une tactique change l'état de preuve (si la tactique fonctionne). Par défaut, la tactique s'applique toujours au premier sous-but, et on ne peut pas agir sur les sous-buts secondaires.

Fin de la preuve. Lorsqu'on arrive à la fin de la preuve, on obtient un λ-terme sans trous. On fait `Qed.`, pieuvre normalise le λ-terme et vérifie qu'il est bien typable avec le type annoncé en début de preuve. Cette dernière étape de normalisation+typage est à l'œuvre en Rocq, elle a pour but de se protéger contre des bugs que pourraient avoir introduit les tactiques.

6.3 Tactiques

On demande de programmer un ensemble de tactiques qui "miment" les règles de la déduction naturelle, ou dans certains cas, des variantes de ces règles. On mimera des tactiques proposées dans l'assistant de preuve Rocq (un concurrent de pieuvre), dont vous pouvez trouver la liste ici.

Comme en Rocq, l'appel à une tactique est suivi d'un point.

Pour la **règle axiome**, implémenter les tactiques `exact` et `trivial`.

`exact`

La tactique `exact` pourra également être utilisée pour fournir une preuve "à la main" directement, sous forme de λ-terme, sans construction interactive.

`trivial`

Introduction de la flèche : les tactiques `intro` et `intros`, avec arguments : `intro x` et `intros x y z`.

`intro`

La tactique `intro` peut échouer si le but à prouver n'est pas une implication. Si on fait `intro x` et `x` est déjà présent dans l'environnement, échec également.

`intros`

Commencez par implémenter `intro`. `intros` utilisera l'implémentation d'`intro`, faites attention au fait qu'`intros x y z` peut échouer "en cours de route", par exemple si on ne peut pas introduire `z`, et il faut alors laisser le but inchangé.

Élimination de la flèche : programmer les tactiques `apply` et `assert`, avec le comportement de ces tactiques en Rocq. La tactique `apply` prend en argument le nom d'une hypothèse, et la tactique `cut` prend en argument un type/une formule (pas de `apply (h2 h5)`, tout du moins dans un premier temps).

`apply`

`cut`

Le faux : programmer les tactiques `exfalso`, `destruct` et `absurd`.

`exfalso`

La première s'utilise sans argument, la deuxième (que l'on n'utilise ici qu'avec une hypothèse prouvant `False`) prend en argument un nom d'hypothèse, la troisième prend en argument une formule.

`destruct`

`absurd`

Ajouter la tactique `admit` qui permet de prouver le sous-but courant en trichant, et qui pourra être utile pour debugger. `admit` pourra correspondre à un constructeur particulier du type `lam`, et engendrer une exception en fin de preuve, au moment de la vérification de type.

`admit`

6.4 Exécutable, session

Votre exécutable s'appellera `pieuvre`. On utilisera les suffixes `.lam` et `.8pus` pour les fichiers ingérés par pieuvre. `pieuvre` proposera les options suivantes :

- `-reduce toto.lam` prend en entrée un λ-terme écrit dans un fichier passé en argument et affiche la suite de ses réductions. Le fichier en entrée contient un λ-terme suivi d'un point.

  Important : comme dit plus haut, bien que les λ-termes contiennent des annotations de typage à chaque `fun`, celles-ci sont ignorées, et on réduit les λ-termes en tant que λ-termes *purs*.

  Important également : soyez conscient.es du fait que quand on fait des β-réductions, il faut potentiellement faire des acrobaties à base d'α-conversion.

- `-alpha toto.lam` prend en entrée deux λ-termes, séparés par le caractère `&` et écrits dans un fichier qui est passé en argument, et affiche `true` ou `false` suivant qu'ils sont α-convertibles ou pas.

  Même remarque que pour la β-réduction : on teste l'α-conversion pour les λ-termes en ignorant les annotations de type (et ainsi, par exemple, l'identité au type A est équivalente à l'identité au type B).

- `-typecheck toto.lam` prend en entrée un fichier contenant un λ-terme et un type, séparés par un `:` et avec un point à la fin, et vérifie que l'on a bien la relation de typage. Contrairement aux deux options précédentes, là, on tient compte des annotations de types !

  On n'utilise l'option `-typecheck` que sur des λ-termes clos, où Γ est vide. Le fichier pourra par exemple contenir `(fun (x:A) => (fun (y:B) => x)) : A -> B -> A.`

- sans option, pieuvre démarre une **session interactive** de l'assistant de preuve.

  - on commence par saisir la propriété que l'on va prouver, en écrivant `Goal`, suivi d'une formule, suivi d'un point.

    On ne met pas les quantifications universelles, autrement dit on écrit

    `Goal A->(A->B)->B.` et pas `Goal forall A B:Prop, A->(A->B)->B.`

  - on saisit les tactiques les unes après les autres, et avec un point à la fin. Celles-ci agissent toujours sur le premier sous-but présenté par pieuvre.

  - on affiche la preuve courante à l'aide de la directive `Show Proof.`. Les parties incomplètes de la preuve sont affichées sous la forme `[?]`.

  - lorsque la preuve est terminée, on affiche un message de félicitations, et après le `Qed.`, on normalise le λ-terme ainsi obtenu, on l'affiche, avec une syntaxe proche de celle de Rocq, et on sort du programme après avoir vérifié que le terme normalisé a le type annoncé pour la preuve (sinon, pieuvre a un bug, et on affiche un message d'erreur..).

    Vous pouvez aussi engendrer au passage un fichier `preuve.lam` contenant le λ-terme normalisé obtenu, voire aussi un fichier `preuve.8pus` qui permet de rejouer la preuve, avec un appel à la tactique `exact`.

  - on n'a pas `Undo`.

- sans option, avec un fichier `.8pus` en argument : on exécute une session comme au point précédent en lisant les entrées successives dans le fichier (chaque entrée est terminée par un point) : la formule à prouver, la suite des tactiques, et `Qed.`.

  Exemple : `pieuvre modusponens.8pus`

  Vous pourrez vous fabriquer une moulinette permettant de faire tourner Rocq sur un fichier `.8pus` : il s'agit d'ajouter le prélude

  ```
  Section pieuvre.
  Context {A B C D E F G : Prop}.
  ```

  en tête de fichier (on a astucieusement freiné à `G`, car ajouter `H` emprunterait l'identifiant standard pour les hypothèses en Rocq), et le postlude

  `End pieuvre.` à la fin, et de lancer la commande `coqc`. Faites-nous signe si vous rencontrez des difficultés pour ce faire.

6.4.1 Exemples d'entrées à soumettre à pieuvre

β-réduction *(les types sont ignorés, raison pour laquelle on se permet des absurdités de types ci-dessous)*

```
(fun (x : A) => x) (fun (y : A) => y) z.

((fun (x : A) => fun (y : A) => y x) u (fun (t:A) => t t).
```

Tests d'α-équivalence.

```
fun (x:A) => fun (y:A) => y & fun (u:A) => fun (v:A) => v.

fun (x:A) => x & fun (y:A) => x.
```

Formules à prouver.

```
(A -> B -> C) -> (B -> A -> C).
A -> B -> A.
A -> (A -> B) -> B.
(A -> B) -> (B -> C) -> A -> C.
((A -> A) -> B) -> B.
(A -> B -> C) -> (A -> B) -> A -> C.
A -> ~(~A).
(A -> B) -> ~B -> ~A.
```

6.5 Enrichir le langage : `/\` `\/` `True`

Remarque : si vous vous sentez d'attaque, vous pouvez sauter cette partie et passer directement à la suivante (types dépendants). Cela est à faire en concertation avec les chargés de TP, avec l'idée qu'il vaut mieux avoir la conjonction et la disjonction qui marchent plutôt que de rater l'ajout des types dépendants parce que c'est trop compliqué.

On enrichit la logique, en reparcourant les diverses étapes du développement de pieuvre décrites jusqu'ici.

On étend le langage des types : $A, B \quad ::= \quad X \mid A \Rightarrow B \mid A \wedge B \mid \top \mid A \vee B \mid \bot$.
En syntaxe ascii, on écrit `/\` `\/` `True`.
Du côté des λ-termes, on ajoute l'unité, les couples et les sommes, ce qui se traduit en ascii par

```
fun x => M    M N    x     (* lambda-calcul *)
exf(M)    (* ex falso quodlibet, l'élimination du faux *)

(M,N)     (* couple *)
fst(M)  snd(M)    (* projections *)
I   (* l'unique terme de type True *)

ig(M,B)  id(M,A)   (* injections, gauche et droite, avec "l'autre type" en argument *)
case(M,N,N')   (* élimination d'une somme *)
```

NB : vous pouvez ignorer le fait que l'on a le droit d'utiliser l'identificateur `I` pour nommer quelque chose dans une preuve, et supposer que l'on ne se trouvera jamais dans ce cas fastidieux.

On étend la β-réduction en projetant les couples et en filtrant les sommes, ce que l'on peut écrire à l'aide des "axiomes ascii" suivants :

```
fst(M,N) --> M
snd(M,N) --> N
case(ig(M,B),N,N') --> N M
case(id(M,A),N,N') --> N' M
```

On fera attention à réduire dans tous les contextes : dans les couples, dans les projections, dans les sommes, dans les filtrages.

On ajoute les tactiques suivantes :

- la tactique `split` pour la conjonction (c'est la règle d'introduction) `split`
- les tactiques `left` et `right` pour la disjonction (ce sont les règles d'introduction) `left` `right`
- la "tactique" `exact I` pour prouver `True` (c'est la règle d'introduction ; on pourrait donner un nom moins abscons, mais c'est ainsi qu'on prouve `True` en Rocq). `exact I`
- la tactique `destruct`, qui prend en argument un nom d'hypothèse. Elle a déjà été introduite pour le cas où l'hypothèse énonce le faux, il faut maintenant gérer les hypothèses qui sont des conjonctions ou des disjonctions, en copiant le comportement de cette tactique en Rocq. À chaque fois on regarde le *connecteur principal* (le constructeur qui est à la racine) de la formule correspondant à l'argument de `destruct`. `destruct`

  Vous pourrez au passage constater qu'on ne retrouve pas systématiquement les règles d'élimination de la déduction naturelle.

Du fait de comment Rocq représente la conjonction et l'implication, il ne sera pas immédiat d'engendrer un λ-terme que l'on peut utiliser comme terme de preuve avec un `exact` : vous pouvez laisser cette question de côté.

6.6 Encore plus loin : types dépendants

Si tout dans ce qui précède est traité/testé/etc., vous pouvez vous référer au chapitre 8 des notes de cours Samuel Mimram, et en particulier la partie 8.5.

On vous suggère d'inclure :

- la quantification universelle et le produit dépendant ;
- un unique type inductif, les entiers (les booléens en plus, éventuellement) ;
- l'égalité (partie 9.1 dans les notes de cours).

L'idée étant d'étendre pieuvre avec la tactique `induction`.

Vous pouvez laisser de côté les univers, quitte à accepter "n'importe quoi" dans la manipulation de `Prop` et `Set`.

Quantification universelle. La première étape consiste à étendre les types pieuvre avec la quantification universelle (dont l'implication est un cas particulier). Les tactiques correspondantes (`intro`, `apply`, ...) seront reprises en conséquence.

On pourra ainsi écrire des preuves comme celles-ci :

```
Goal forall A B: Prop, A/\B -> B/\A.

intros. destruct H; split; trivial.
Save toto.


Goal forall A:Set, forall R: A->A->Prop,
  (forall x y, R x y -> R y x) ->
  (forall x y z, R x y -> R y z -> R x z) ->
  forall a b c, R a b -> R b c -> R c a.
```

```
intros A R sym trans a b c.
intros.
apply sym.
apply (trans a b).
trivial.
trivial.
Save titi.
```

Égalité. Pour avoir des intutions sur comment marche l'égalité, vous pouvez jouer avec des preuves en Rocq, et afficher les termes de preuve ainsi construits. Ainsi par exemple :

```
Lemma plus_n_O : forall n, n+0 = n.

induction n; simpl; intros; trivial.
rewrite IHn; trivial.
Qed.

Print plus_n_O.

Print eq_ind_r.

Lemma rewr: forall A: Set, forall a b c:A, a=b -> b=c -> a=c.

intros A a b c H H'.
rewrite H; assumption.
Qed.

Print rewr.

Print eq_ind_r.

Print eq_sym.

Print eq_ind.

Print eq.
```